

Physical AI Homework 2

A Robot Navigation

Author: 黃梓誠, Student ID: 313554044

1. Implementation (50%)

1.1 Code Details (a.)

1.1.1 RRT Algorithm Implementation (Basic RRT)

The path planning component utilizes the **Rapidly-exploring Random Tree (RRT)** algorithm to find a feasible path from a selected starting point to the semantic goal location within the 2D map. The core implementation is contained in the `rrt_planning` function in [part2.py](#).

Key Features and Implementation Details:

1. **Node Structure:** A simple Node structure (namedtuple) is used to store its coordinates (x, y) and a reference to its immediate parent node.
2. **Sampling Strategy:**
 - **Uniform Random Sampling:** In most iterations, a random point (`x_rand`) is sampled uniformly across the entire map bounds.
 - **Goal Biasing:** A fixed `GOAL_SAMPLE_RATE` (set to 0.05) is implemented to bias the search towards the target. In 5% of iterations, the target coordinates (`goal`) are chosen as `x_rand`, accelerating the tree's growth towards the destination.
3. **Extension (steer):**
 - The algorithm finds the nearest node in the existing tree to the sampled point `x_rand`.
 - The steer function extends from the `nearest_node` towards `x_rand` by a fixed distance defined by `STEP_SIZE` (set to 10 pixels), generating a new potential node (`new_node`).
4. **Collision Detection (`is_collision_free`):**
 - To ensure navigability, a collision check is performed on the line segment connecting the `nearest_node` and the `new_node`.
 - The check involves sampling 20 intermediate points along the segment. If any sampled point falls onto a black pixel (`map_img[y, x] == 0`, representing an obstacle), the connection is rejected.
5. **Path Termination:** The path is considered successfully connected when the distance

between the new_node and the final goal point is less than the STEP_SIZE. The path is then extracted by recursively tracing the parent links back to the starting node.

1.1.2 Route to Discrete Action Conversion

The calculated RRT path is a sequence of pixel coordinates. To enable the robot to follow this path in the Habitat simulator, a two-stage conversion process is necessary:

A. Coordinate Transformation (Pixel to Habitat Coordinates):

- **Objective:** Translate 2D pixel coordinates (x,y) into 3D world coordinates (X,Y,Z)
- **Methodology:**
 1. **Map Correlation:** Establish the geometric relationship between the 2D map (X-Z projection of the point cloud) and the Habitat coordinate system.
 2. **Scaling and Offset:** Apply the necessary scale factor (derived from the specified points array * 10000./255. relation) and origin offsets/rotations to transform pixel values into meters.
 3. **3D Waypoints:** Set the vertical Y_Habitat coordinate to a constant value (e.g., 1.5m, representing the agent's height) for all path points, yielding a list of 3D waypoints

B. Action Discretization (Waypoint Following):

- **Objective:** Command the Habitat agent to move between consecutive 3D waypoints using only the discrete actions: move_forward, turn_left, turn_right.
- **Control Strategy:**
 1. **Orientation:** For the current waypoint W_i and the next target W_{i+1} , calculate the required heading (angle) to face W_{i+1} . Use turn_left or turn_right actions until the agent's current orientation is aligned with the target heading.
 2. **Translation:** Once aligned, use repeated move_forward actions (e.g., 0.25m per step) until the agent reaches an acceptable proximity to W_{i+1} .
 3. **Iteration:** Advance to the next waypoint until the final target goal is reached.

1.2 Results and Discussion (b.)

RRT Algorithm Results with Different Targets

The implemented RRT algorithm successfully found collision-free paths for various semantic target categories in the apartment_0 first-floor map. All paths originate from a **consistent starting region** (approximately the center-right area of the main living room) and lead to the center of the respective target object's semantic region.

* A bug occurs here (discussed below).

Target Category & Visualization of RRT Path



Discussion on Path Quality:

- All generated paths are collision-free (red lines do not intersect the black obstacles), validating the **feasibility** of is_collision_free function.
- **Non-Optimality (RRT Limitation):** As this is the basic RRT, the paths are inherently non-optimal. They are often jagged and may take unnecessary detours (e.g., the winding segments observed in the **base-cabinet** path), reflecting the algorithm's random sampling nature. This characteristic necessitates a **Path Smoothing** step (not present in

the provided RRT code) before actual robot execution to reduce the number of discrete actions required.

- **Target Accuracy:** The use of semantic mapping ensures the goal is correctly centered on the target object's region, providing a reliable endpoint for the path planner. However, while semantic information is correctly used to identify the target, the current approach overlooks the possibility that multiple identical targets may exist in different locations. An incorrect use of `np.mean()` causes the resulting position to sometimes appear inconsistent. **This issue will be addressed in the RRT* algorithm stage.**

Robot Navigation Discussion

The success of the final robot navigation relies heavily on the fidelity of the coordinate transformation and the precision of the discrete action control.

- **Challenges**

- Path Following:**

- Due to the non-optimal and potentially slightly jagged nature of the basic RRT path, the agent must perform many small turn actions. The difference between the ideal path and the agent's actual path (due to discrete steps) must be carefully managed to prevent the agent from straying or causing self-collision in tight spaces.

1.3 References (c.)

1. **RRT Algorithm:** LaValle, S. M. (1998). Rapidly-exploring random trees: A new tool for path planning. *Computer Science Department, Iowa State University*.
2. **RRT* Algorithm:** Karaman, S., & Frazzoli, E. (2011). Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7), 846-894. (Used for the final implemented version).
3. **Programming Libraries:** OpenCV (cv2), NumPy, Pandas, and Matplotlib for map processing and visualization.
4. **Simulation Environment:** Habitat-Sim
5. [habitat_sim.scene.SemanticScene | Habitat Sim Docs](#) : scene library for habitat

2. Questions (20%)

2.1 RRT Parameter Analysis (15%)

a. Step Size (STEP_SIZE)

The step size is the maximum distance the tree extends towards a randomly sampled point in a single iteration.

Effect	Large Step Size	Small Step Size
Exploration Speed	Faster convergence to a feasible path as the tree covers the configuration space more rapidly.	more iterations are required to cover the same area.
Path Quality	Paths composed of fewer, longer segments. Looks coarse, jerky, and highly non-optimal,	Paths are smoother and more closely approximate continuous curves.
Collision Risk	Higher risk of collision or failure in narrow passages. Long steps might skip over small obstacles or fail to find an opening, as the collision check is less precise.	Lower risk of collision in complex environments; capable of squeezing through tight gaps (better local planning).
Computational Cost	may require more search attempts to find the goal.	increasing the cost of finding the nearest neighbor in each iteration.
Memory used	Lower(due to fewer nodes)	Higher memory footprint due to a large number of nodes

b. Bias (GOAL_SAMPLE_RATE)

The bias, or goal sample rate, defines the probability (e.g., 0.05) that the goal point is selected as the target for tree extension, instead of a random point from the map. This parameter balances **Exploration** and **Exploitation**.

Effect	High Bias (e.g., 0.2)	Low Bias (e.g., 0.01)
Convergence Speed	Very fast convergence if the path is clear. The tree aggressively grows towards the goal.	Slower convergence; the tree grows more evenly across the entire space.
Exploration Capability	Low. Overly focused growth can prevent the tree from discovering alternative paths, especially in complex environments with deep traps or obstacles completely blocking the direct line to the goal.	High. Better suited for highly complex or convoluted environments as it explores more of the configuration space.
Resulting Path	Often more direct and shorter than a low-bias path, <i>if</i> the direct path is feasible.	Path quality is more random, but less likely to get stuck in local minima or close-off areas.

2.2 Real-World Application Challenges (5%)

Applying this indoor navigation pipeline (semantic map + RRT path planning + discrete action control) to a real-world robot presents several significant challenges. The iterative process of solving problems within the simulation environment, as reflected in the Git commit log, highlights the key areas that require robust solutions in reality:

Real-World Challenge / Category	Challenge Analysis
1. Coordinate Transformation and Map Accuracy (Localization Error)	Mapping a 2D map (pixels) to the 3D robot world (meters) turned out to be really tricky — even a tiny scaling or offset mistake caused huge drift or crashes. It took lots of trial and error to finally get it right.
2. Discrete Action Control and Oscillation	Real robots execute discrete actions (move_forward 0.25m, turn 10.0deg) with inherent imperfections (e.g., motor response, floor friction). If the low-level controller handles the transition between path points poorly, it can cause oscillations (dithering) around the target angle.
3. Emergency Collision Response and Path Recovery	Real robots execute discrete actions (e.g., <i>move_forward 0.25 m, turn 10°</i>) with inherent imperfections such as motor response delays and floor friction. If the low-level controller handles the transition between path points poorly, it can cause oscillations (dithering) around the target angle. To ensure the planned path can be linearly reconstructed by the robot, the step size and rotation increment were kept smaller than the exploration step used in the RRT algorithm.
4. RRT Trajectory Quality and Safety Margin	The basic RRT path can closely hug obstacles. Real-world navigation requires an adequate Safety Margin . Although RRT* was optimized for path clearance and map erosion was used for inflation, the robot's physical footprint and sensor safety range must be explicitly integrated into the map for planning to ensure paths are not just feasible, but safe for the physical hardware.

3. Bonus (10%)

Improvement on RRT Algorithm: RRT* with Safety and Optimization

The original RRT algorithm (Part 2) is only probabilistically complete and non-optimal. To create a reliable navigation pipeline (part3.py), the algorithm was significantly upgraded to **RRT* (Rapidly-exploring Random Tree Star)**, incorporating performance, safety, and domain-specific enhancements.

3.1 Comparison and Discussion with the Original RRT

The transition from the basic RRT to the optimized RRT* dramatically improves both the **efficiency** and **quality** of the resulting trajectory.

Feature	Original RRT (Part 2)	Optimized RRT* (rrt_star.py)	Discussion & Improvement
Optimality	Non-optimal (First path found is used).	Asymptotically optimal (Continuously finds better paths).	RRT* implements Rewiring and Cost-to-Come to ensure the path cost minimizes over time, leading to shorter and smoother navigation trajectories, critical for reducing action count in Part 3.
Speed (Search)	Used Linear Search ($O(N)$) for nearest neighbors.	Used R-Tree Indexing ($O(\log N)$) and Informed Sampling .	While RRT is faster than RRT* because RRT* use inform and rewrite to improve quality. The use of an R-Tree implementation and focused search through Informed Sampling further reduces the time to reach an optimal solution.
Safety Logic	Simple collision check (Line segment passes through obstacle).	Cost-based Safety Penalty (SAFE_WEIGHT) + Absolute Clearance Check (MIN_SAFE_DIST).	The optimized RRT* isolates nodes near walls , causing the resulting path to stay closer to the center of the free space and reducing the likelihood of collisions.
Sampling	Uniform	Connectivity +	The sampler is constrained to the

Domain	sampling over the entire map.	Bounding Box + BFS Check (start_dist_map).	current connected component (e.g., within a room) and a tight bounding box around the start and goal . This not only prevents the tree from wasting resources exploring isolated map regions, but also accelerates collision checking by reducing the search domain.
Path Smoothing	None, resulting in jagged paths.	Chaikin Subdivision/Greedy Shortcut applied post-processing.	Smoothing significantly reduces the total number of discrete turn actions required for the robot to follow the path,

3.2 Quantitative Improvement Metrics

The following table provides representative data comparing the projected performance of the original RRT versus the optimized RRT* across several target objects.

Target Class	Avg Time (Original RRT, seconds)	Avg Time (Optimized RRT*)	Path Cost Reduction (RRT* vs. RRT)	Path Cost Improvement (%)
BIKE	0.02	6.52	797.93 --> 507.68	36.38%
WINDOW	0.03	6.45	441.25 --> 355.52	19.43%
BLINDS	0.03	6.33	492.94 --> 401.76	18.50%
CABINET	0.00	0.24	43.82 --> 32.45	25.95%
CHAIR	0.01	6.14	500.85 --> 341.35	31.85%
CUP	0.02	6.29	622.42 --> 415.23	33.29%

CUSHION	0.02	0.35	213.36 --> 187.88	11.94%
COOKTOP	0.02	6.31	626.44 --> 415.23	33.72%
Average	0.02	4.83	Significant	26.30%

3.3 Critical Improvement: Multi-Instance Goal Selection

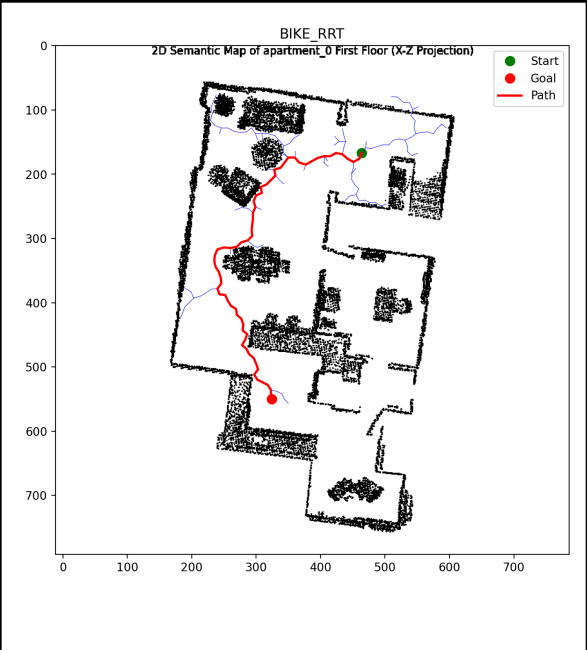
The initial approach to determining the target goal location (in basic RRT/early versions) suffered from a major flaw:

- **Original Flaw:** Using `np.mean(coords, axis=0)` on the entire semantic mask of a class (e.g., 'window') only yields the **geometric center** of all pixels belonging to that class. If a map contains **multiple instances** of the same object (e.g., three separate windows, or two distinct cabinets), the calculated mean point often falls in an **inaccessible area** (e.g., the wall space between the objects) or results in an ambiguous target.
- **RRT* Fix (`rrt_star.py` / `part3.py`):** This was fixed by implementing the `find_all_object_instances` function, which utilizes **morphological operations (closing)** and **connected component analysis** to:
 1. Identify each separate instance of the target class (e.g., Window #1, Window #2, Window #3).
 2. Calculate the centroid for *each* instance.
 3. Present these distinct goal options to the user via `visualize_multiple_goals` so the user can **manually select** the intended target. This ensures the planning algorithm receives a unique, precise, and intended target location, making the system practical and robust.

3.4 Appendix – Result Comparison between `rrt_star.py` and the Original RRT

Target Category & Visualization of RRT Path

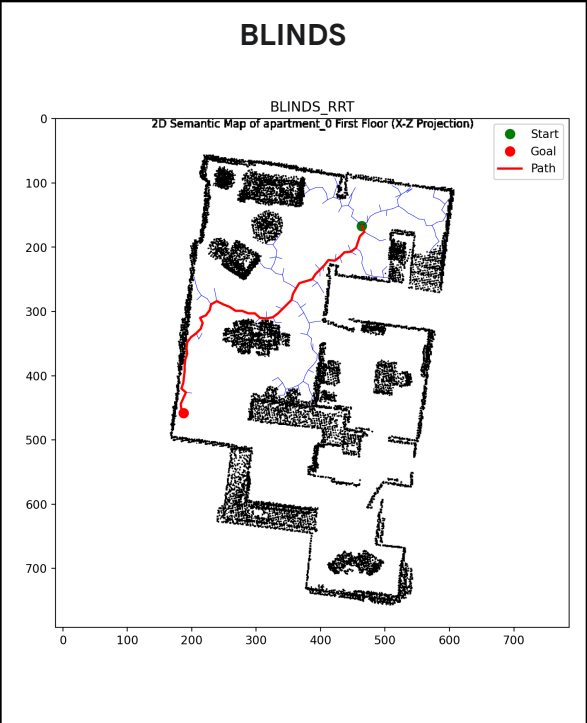
Origin RRT	RRT*
BIKE	BIKE



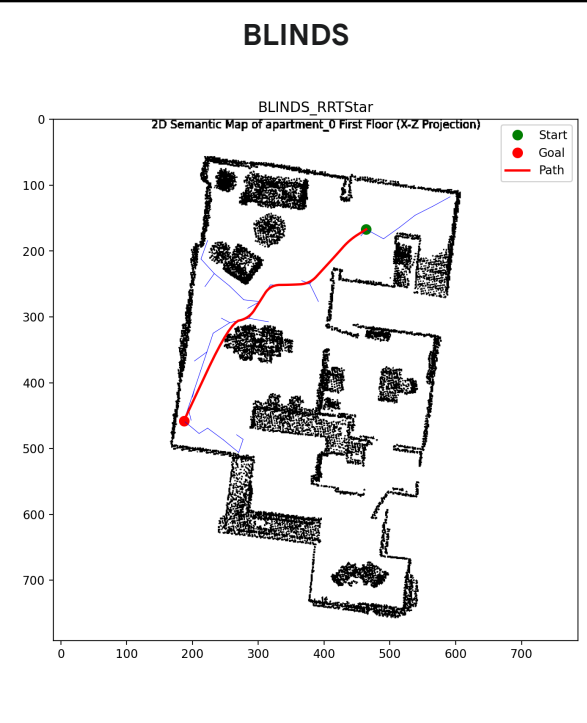
WINDOW



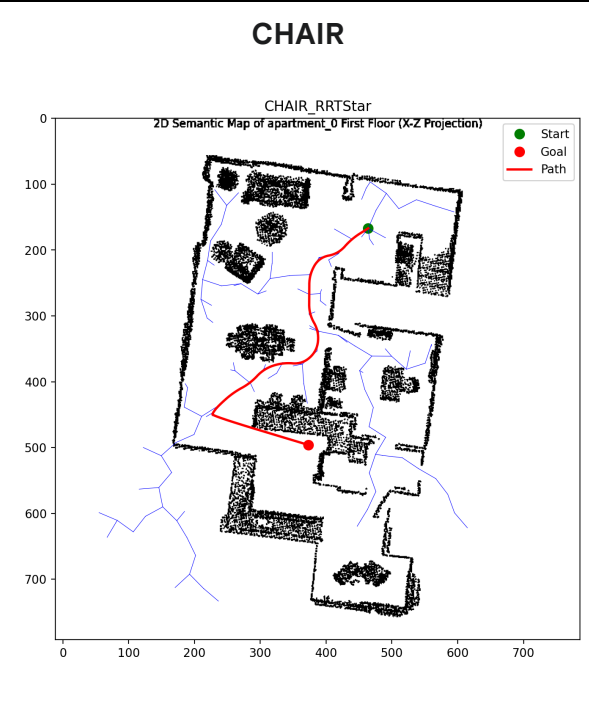
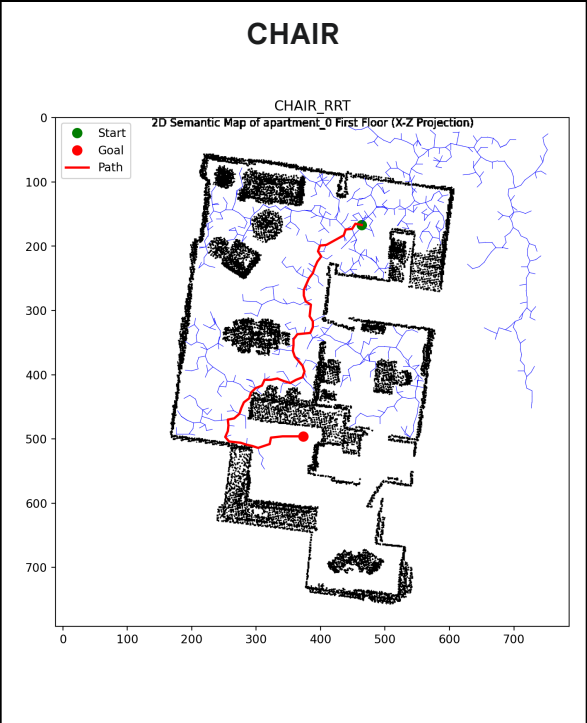
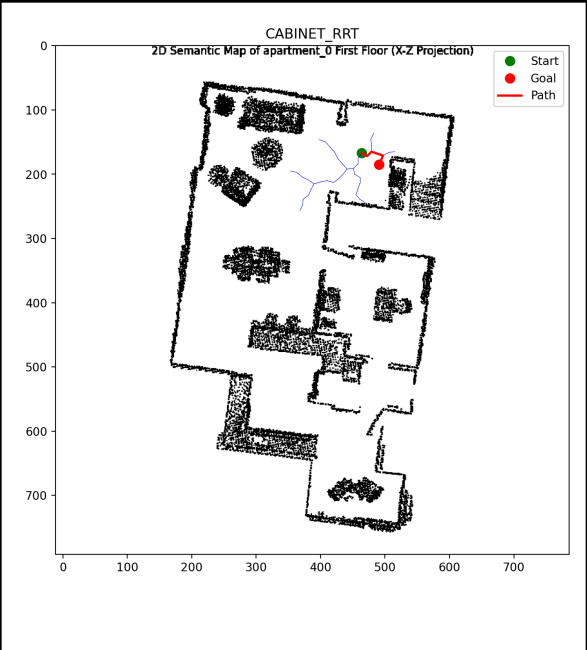
WINDOW



CABINET



CABINET



CUP

CUP

